MODUL PERTEMUAN 5

PEMROGRAMAN MOBILE (SIF65005)

Form Input dan Validasi untuk Menjamin Kualitas Data

Proyek Semester: form tambah data mahasiswa

Program Studi S1 Sistem Informasi
Fakultas Sains dan Teknologi
UIN Sulthan Thaha Saifuddin Jambi

Catatan untuk Dosen (hapus sebelum dibagikan)

Modul ini disusun dari RPS SIF65005, Minggu 5. Bagian berikut **perlu Bapak periksa atau lengkapi**:

Hal	Keputusan di modul ini	Tindakan dosen
Bobot 3%	RPS mencantumkan teknik "quiz, tugas praktik" dengan bobot 3%, tanpa rincian pembagian.	Isi pembagian di Bab 12 (ditandai [....]).
Bobot aspek rubrik Tugas 4	Bobot aspek (35/25/20/10/10) adalah usulan penyusun.	Sesuaikan bila perlu.
Batas cakupan	Data yang valid **belum** dimasukkan ke daftar Dashboard. Mengirim data hasil input dan memperbarui daftar adalah materi RPS Pertemuan 6. Di P5, data valid ditampilkan sebagai pratinjau dan SnackBar.	—
Aturan NIM dan semester	Contoh memakai NIM minimal 7 digit dan semester 1–14.	Sesuaikan dengan format NIM dan peraturan akademik kampus.
Pembatas ketikan	`FilteringTextInputFormatter` berasal dari pustaka bawaan Flutter (`package:flutter/services.dart`), bukan package tambahan.	—
Widget pendukung	Selain materi RPS, kode memakai SnackBar, FloatingActionButton.extended, OutlinedButton, dan InputDecoration.	Hapus bila ingin ketat.
Pesan error	Ditulis dari format umum Dart/Flutter. Teks bisa sedikit berbeda antarversi.	Cocokkan dengan hasil di lab.
`flutter analyze`	Penyusun **tidak dapat menjalankan** Flutter. Kode Bab 7 belum diuji.	Uji di lab (lihat Bab 7.6).

1. Identitas

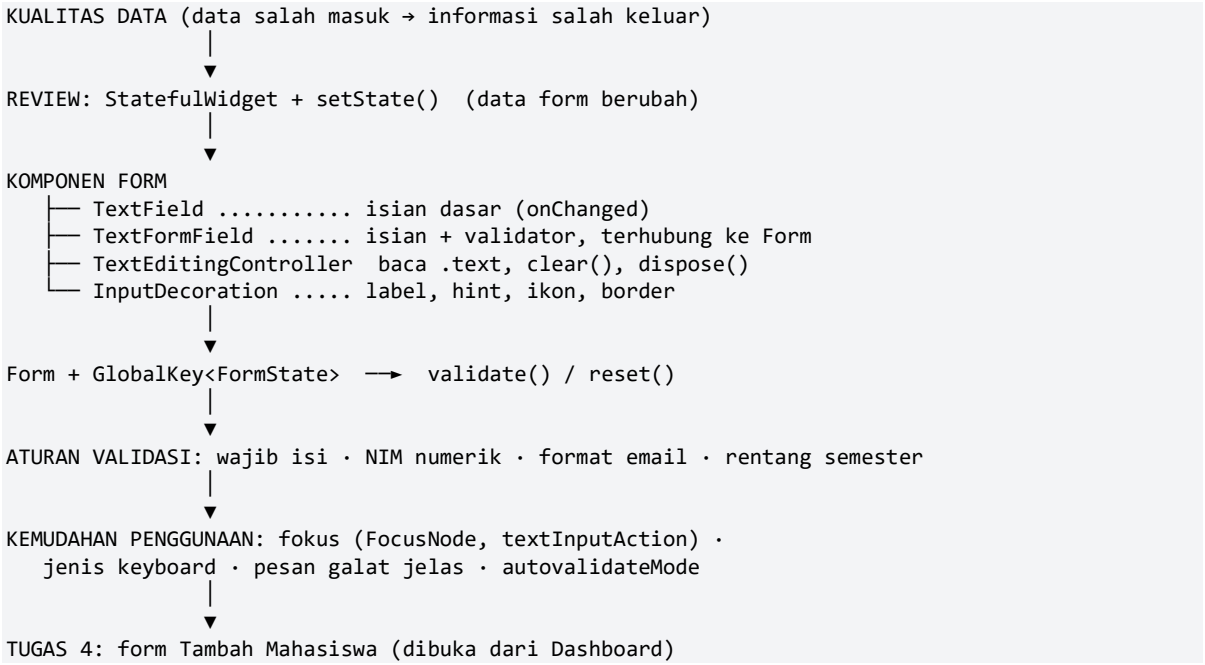
Mata kuliah	Pemrograman Mobile
Kode / SKS / Semester	SIF65005 / 4 SKS / V
Pertemuan	Minggu ke-5
Topik	Form input dan validasi
Durasi	KPB 200’; KPT 80’; KM 80’
CPL	CPL04 – Mampu merancang, mengembangkan, dan mengevaluasi aplikasi sistem informasi berbasis web/mobile sesuai kebutuhan pengguna dan standar kualitas perangkat lunak.
CPMK	CPMK042 – Mengembangkan aplikasi mobile dengan mempertimbangkan kualitas perangkat lunak
Sub-CPMK	Sub-CPMK36111 – Mahasiswa mampu merancang antarmuka aplikasi mobile berdasarkan kebutuhan pengguna secara tepat.
Kemampuan akhir Minggu 5	Sub-CPMK36111 (penguatan): merancang form input dengan validasi untuk menjamin kualitas data masukan.
Indikator	Menggunakan TextFormField dan TextEditingController; membangun Form dengan GlobalKey<FormState>; menulis aturan validasi (wajib isi, NIM numerik, format email); mengelola fokus dan pesan galat.
Kriteria & teknik penilaian	Kriteria: kelengkapan aturan validasi dan kemudahan penggunaan. Teknik: quiz, tugas praktik.
Bentuk, metode, penugasan	Live coding; PjBL. Tugas 4: form tambah data mahasiswa.
Materi	TextField dan TextFormField; TextEditingController; Form dan GlobalKey<FormState>; validator; pengelolaan fokus; review StatefulWidget dan setState(). [1][4]
Bobot	3%
Dosen pengampu	.....

2. Tujuan Pembelajaran dan Indikator Capaian

Setelah mengikuti Pertemuan 5, mahasiswa mampu:

Kode	Tujuan pembelajaran	Level Bloom	Indikator RPS	Bukti capaian
T1	Menjelaskan perbedaan StatelessWidget dan StatefulWidget, serta peran `setState()`.	C2 Memahami	(Materi: review StatefulWidget dan setState())	Kuis 1; demo sapaan
T2	Menggunakan `TextFormField` dan `TextEditingController` untuk membaca, mengatur, dan mengosongkan isian.	C3 Menerapkan	Menggunakan TextFormField dan TextEditingController	Tugas 4
T3	Membangun `Form` dengan `GlobalKey<FormState>` dan memanggil `validate()` serta `reset()`.	C3 Menerapkan	Membangun Form dengan GlobalKey<FormState>	Tugas 4; kuis 2
T4	Merancang dan menulis aturan validasi wajib isi, NIM numerik, dan format email, beserta pesan galat yang jelas.	C6 Mencipta	Menulis aturan validasi (wajib isi, NIM numerik, format email)	Tabel aturan; kuis 3; latihan
T5	Mengatur urutan fokus, jenis keyboard, dan waktu munculnya pesan galat agar form mudah dipakai.	C3 Menerapkan	Mengelola fokus dan pesan galat	Tugas 4; peer review
T6	Menganalisis penyebab form yang gagal disimpan atau menampilkan pesan yang membingungkan.	C4 Menganalisis	(Kriteria: kelengkapan validasi dan kemudahan penggunaan)	Kuis 4–5

3. Peta Konsep



4. Prasyarat dan Persiapan

4.1 Pengetahuan yang dipakai

Asal	Pengetahuan	Dipakai untuk
Pertemuan 2	Counter (`StatefulWidget`, `setState`), Hot Reload vs Hot Restart	Review state
Pertemuan 3	Fungsi, percabangan, null safety (`?`, `??`), Map, `BarisInfo`	Menulis validator dan pratinjau
Pertemuan 4	`Navigator.push`, Dashboard	Membuka form dari Dashboard
Basis Data (bekal tambahan, di luar MK syarat RPS)	Integritas data, tipe kolom, *constraint* (batasan)	Memahami alasan validasi

4.2 Checklist sebelum kuliah

- Proyek `aplikasi_akademik` hasil Tugas 3 (Dashboard–Profil–Detail) bisa dijalankan.
- Sketsa halaman "Tambah Mahasiswa" dan daftar aturan isian (tugas persiapan P4).
- Catatan satu contoh form di aplikasi lain dan cara aplikasi itu menampilkan pesan galat.
- Peramban untuk DartPad (demo review StatefulWidget).
- Tidak ada package tambahan. Semua widget form tersedia di Flutter.

5. Apersepsi

Analogi: petugas loket memeriksa formulir pendaftaran.

Anda menyerahkan formulir wisuda di loket akademik. Petugas langsung memeriksa: ada kolom kosong? NIM berisi huruf? Email tidak lengkap? Petugas melingkari kolom yang salah dengan pulpen merah dan menulis catatan, lalu mengembalikan formulir untuk diperbaiki. Formulir baru diterima bila semua kolom benar.

Di loket akademik	Di Flutter
Lembar formulir	`Form`
Kolom isian	`TextFormField`
Tulisan Anda di kolom	`TextEditingController.text`
Daftar aturan yang dipegang petugas	`validator` setiap kolom
Petugas memeriksa seluruh formulir	`_formKey.currentState!.validate()`
Lingkaran dan catatan merah	Pesan galat di bawah kolom
Formulir diterima	`validate()` bernilai `true`

Pertanyaan pemantik: Apa akibatnya bila sistem akademik menyimpan NIM "2023a01" atau email "rina@"? Siapa yang dirugikan?

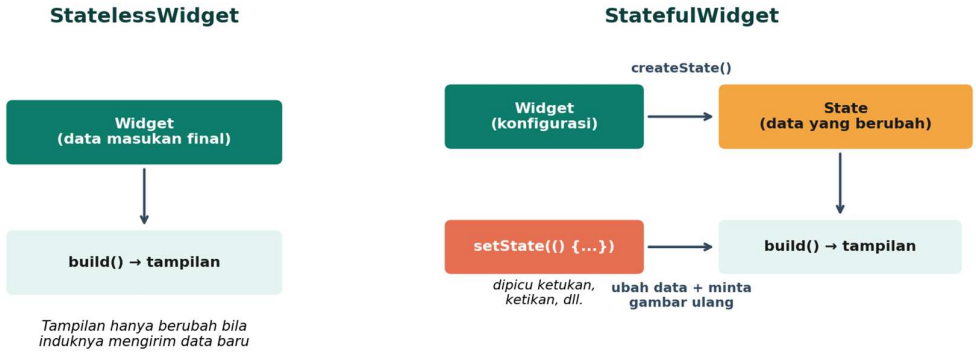
> Prinsip sistem informasi: *garbage in, garbage out* (data sampah masuk, informasi sampah keluar). Validasi adalah pintu pertama menjaga kualitas data.

6. Uraian Materi

Materi disusun bertahap: **dasar** (6.1–6.3) → **menengah** (6.4–6.6) → **aplikatif** (6.7–6.9).

6.1 [Dasar] Review StatefulWidget dan setState()

State (keadaan) adalah data yang bisa berubah selama halaman tampil, misalnya angka counter, isi form, atau hasil validasi.



Gambar 6.1 StatelessWidget dan StatefulWidget

	StatelessWidget	StatefulWidget
Data	Hanya data masukan (final)	Data masukan + state yang berubah
Jumlah class	Satu	Dua: widget dan `State<...>`
Menggambar ulang	Saat induk mengirim data baru	Saat `setState()` dipanggil
Contoh di proyek	`ProfilPage`, `DetailMahasiswaPage`, `BarisInfo`	Counter (P2), `TambahMahasiswaPage` (P5)

Aturan setState(): ubah data **di dalam** `setState(() { ... })`. Flutter lalu memanggil `build()` lagi sehingga layar menampilkan data terbaru. Mengubah data di luar `setState` tidak memperbarui layar.

Siklus hidup singkat:

Method	Kapan dipanggil	Dipakai untuk
`initState()`	Sekali, saat State dibuat	Menyiapkan data awal
`build()`	Setiap kali perlu menggambar (termasuk setelah `setState`)	Mengembalikan tampilan
`dispose()`	Sekali, saat halaman ditutup	Melepas controller dan FocusNode agar memori tidak bocor

Demo review (DartPad): ketikan langsung menjadi sapaan.

```
// Review StatefulWidget: sapaan langsung saat mengetik (jalankan di DartPad)
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: SapaPage()));

class SapaPage extends StatefulWidget {
  const SapaPage({super.key});
```

```
  @override
  State<SapaPage> createState() => _SapaPageState();
}

class _SapaPageState extends State<SapaPage> {
  String _nama = ''; // data yang berubah (state)

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Review StatefulWidget')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              decoration: const InputDecoration(labelText: 'Ketik nama Anda'),
              // Setiap huruf diketik: simpan ke state, lalu gambar ulang.
              onChanged: (teks) => setState(() => _nama = teks),
            ),
            const SizedBox(height: 16),
            Text(
              _nama.isEmpty ? 'Halo!' : 'Halo, $_nama!',
              style: const TextStyle(fontSize: 22),
            ),
          ],
        ),
      ),
    );
  }
}
```

6.2 [Dasar] TextField dan TextFormField

	TextField	TextFormField
Fungsi	Kolom isian dasar	Kolom isian yang menjadi bagian dari `Form`
Validasi	Tidak punya `validator`	Punya `validator` dan menampilkan pesan galat
Membaca isi	`controller` atau onChanged	`controller`, onChanged, atau onSaved
Cocok untuk	Kolom pencarian, isian tunggal	Form data yang harus valid

Keduanya memakai `decoration: InputDecoration(...)` untuk label, petunjuk, dan ikon.

Properti InputDecoration	Fungsi	Contoh
`labelText`	Nama kolom; naik ke atas saat diisi	`'NIM'`
`hintText`	Contoh isian saat kolom kosong	`'hanya angka'`
`helperText`	Keterangan tetap di bawah kolom	`'Sesuai KTM'`
`prefixIcon`	Ikon di kiri kolom	`Icon(Icons.badge)`
`border`	Bentuk garis kolom	`OutlineInputBorder()`

Properti lain yang membantu pengguna:

Properti TextFormField	Fungsi
`keyboardType`	Jenis keyboard: `TextInputType.number`, `.emailAddress`, `.phone`, `.text`
`textInputAction`	Tombol aksi keyboard: `next` (pindah kolom) atau `done` (selesai)
`textCapitalization`	Huruf besar otomatis, mis. `words` untuk nama
`maxLength`	Batas jumlah karakter, beserta penghitungnya
`inputFormatters`	Menyaring ketikan, mis. `FilteringTextInputFormatter.digitsOnly` (perlu `import 'package:flutter/services.dart';`)
`obscureText`	Menyamarkan isian, untuk kata sandi (dipakai di P13)

6.3 [Dasar] TextEditingController

TextEditingController (pengendali teks) adalah objek yang memegang isi sebuah kolom. Lewat controller, kode bisa membaca dan mengubah isian.

```
final _nimController = TextEditingController();  // 1. buat (di State, bukan di build)

TextFormField(controller: _nimController)          // 2. hubungkan

final nim = _nimController.text.trim();            // 3. baca (trim: buang spasi)
_nimController.text = '2023001';                  // 4. isi dari kode
_nimController.clear();                           // 5. kosongkan

@override
void dispose() {
  _nimController.dispose();                       // 6. lepaskan saat halaman ditutup
  super.dispose();
}
```

> Controller dibuat **sekali** sebagai field di class State. Bila dibuat di dalam `build()`, controller baru tercipta setiap kali layar digambar ulang dan isian bisa hilang.

6.4 [Menengah] Form dan GlobalKey<FormState>

Form membungkus beberapa `TextFormField` agar bisa diperiksa sekaligus. Untuk memerintah Form dari tombol, dibutuhkan *GlobalKey* (kunci global) bertipe `FormState`.

```
final _formKey = GlobalKey<FormState>();   // di class State

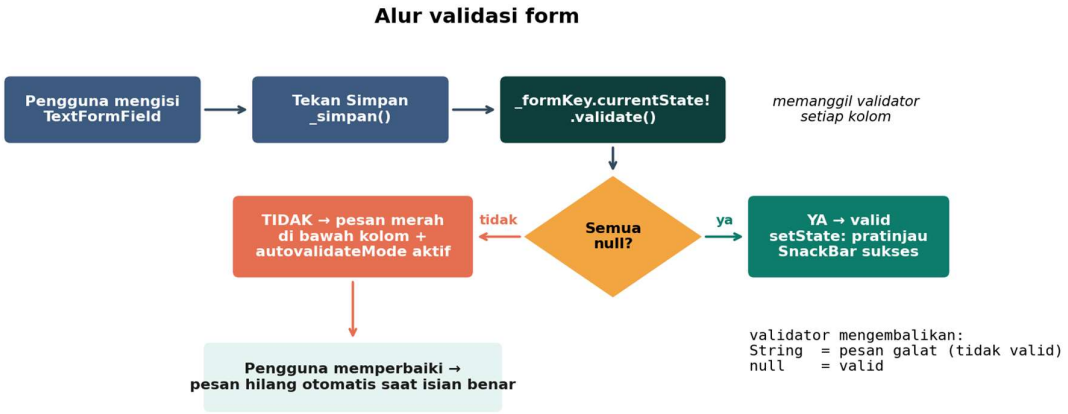
Form(
  key: _formKey,                           // pasang kunci
  child: Column(children: [ TextFormField(...), ... ]),
)

// di tombol Simpan
if (_formKey.currentState!.validate()) {
  // semua validator mengembalikan null → data valid
}

_formKey.currentState!.reset();          // kembalikan ke nilai awal
```

Bagian	Arti
`GlobalKey<FormState>()`	Kunci unik untuk mengakses state Form dari luar widget Form.
`currentState!`	State milik Form. Tanda `!` karena bisa null bila kunci belum dipasang.

Bagian	Arti
`validate()`	Memanggil semua validator, menampilkan pesan galat, mengembalikan `true` bila semuanya valid.
`reset()`	Mengembalikan semua kolom ke nilai awal dan menghapus pesan galat.



Gambar 6.2 Alur validasi form

6.5 [Menengah] Validator: Aturan dan Pesan Galat

Validator adalah fungsi yang menerima isian kolom dan mengembalikan:

- **String** berisi pesan galat bila isian salah, atau
- **null** bila isian valid.

Validator ditulis sebagai fungsi terpisah agar mudah dibaca dan diuji.

a. Wajib isi

```
String? validasiNama(String? nilai) {
  final teks = nilai?.trim() ?? '';
  if (teks.isEmpty) return 'Nama wajib diisi';
  if (teks.length < 3) return 'Nama minimal 3 huruf';
  return null;
}
```

`trim()` membuang spasi di awal dan akhir, sehingga isian berisi spasi saja tetap dianggap kosong.

b. NIM numerik dengan RegExp

RegExp (*regular expression*, ekspresi reguler) adalah pola untuk mencocokkan teks.

```
if (!RegExp(r'^[0-9]+$').hasMatch(teks)) {
  return 'NIM hanya boleh berisi angka';
}
```

Bagian pola	Arti
`r'...'`	*Raw string*: garis miring terbalik dibaca apa adanya
`^` dan `$`	Awal dan akhir teks: seluruh teks harus cocok
`[0-9]`	Satu digit 0 sampai 9
`+`	Satu atau lebih dari pola sebelumnya

`FilteringTextInputFormatter.digitsOnly` mencegah huruf diketik. Validator tetap dipasang sebagai pengaman, misalnya bila teks ditempel (*paste*).

c. Format email

```
final polaEmail = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');
if (!polaEmail.hasMatch(teks)) {
  return 'Format email tidak valid (contoh: nama@kampus.ac.id)';
}
```

Bagian pola	Arti	Cocok dengan
`[\w.+-]+`	Huruf, angka, garis bawah, titik, plus, minus (satu atau lebih)	`rina.amelia`
`@`	Tanda @ wajib ada	`@`
`[\w-]+`	Nama domain	`contoh`
`(\.[\w-]+)+`	Satu atau lebih ".bagian"	`.ac.id`

> Pola ini sengaja sederhana untuk pembelajaran. Validasi email sempurna sangat rumit; cara paling pasti adalah mengirim email verifikasi (dibahas di autentikasi P13).

d. Rentang angka

```
final angka = int.tryParse(teks);              // null bila bukan angka
if (angka == null) return 'Semester harus berupa angka';
if (angka < 1 || angka > 14) return 'Semester harus antara 1 dan 14';
```

e. Pesan galat yang baik

Kurang baik	Lebih baik	Alasan
"Error"	"NIM wajib diisi"	Menyebut kolom dan masalahnya
"Input tidak valid"	"NIM hanya boleh berisi angka"	Menjelaskan aturan yang dilanggar
"Email salah!!!"	"Format email tidak valid (contoh: nama@kampus.ac.id)"	Memberi contoh perbaikan, nada sopan
Semua pesan muncul sebelum mengetik	Pesan muncul setelah Simpan, lalu diperbarui saat mengetik	Tidak memarahi pengguna yang belum mulai

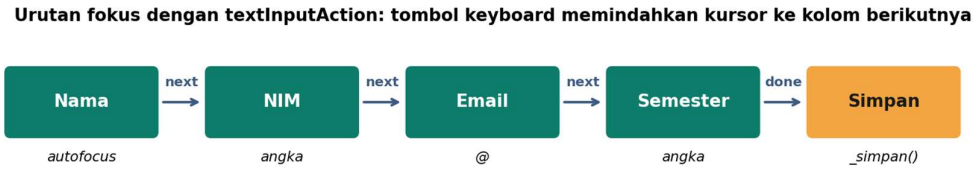
6.6 [Menengah] Kapan Pesan Galat Muncul: autovalidateMode

Nilai	Perilaku	Kesan bagi pengguna
`AutovalidateMode.disabled`	Pesan hanya muncul saat `validate()` dipanggil	Tenang; cocok sebelum Simpan pertama
`AutovalidateMode.onUserInteraction`	Pesan diperbarui setiap kali kolom diubah	Umpan balik langsung saat memperbaiki
`AutovalidateMode.always`	Pesan muncul sejak halaman dibuka	Mengganggu; jarang dipakai

Strategi di contoh kode: mulai dengan `disabled`. Bila Simpan pertama gagal, ubah menjadi `onUserInteraction` lewat `setState`, sehingga pesan hilang begitu isian diperbaiki.

6.7 [Aplikatif] Mengelola Fokus

Focus (fokus) adalah kolom yang sedang aktif menerima ketikan. Pengelolaan fokus membuat pengisian cepat tanpa harus mengetuk setiap kolom.



Gambar 6.3 Urutan fokus pada form Tambah Mahasiswa

Teknik	Kode	Efek
Fokus otomatis	`autofocus: true`	Kursor langsung di kolom Nama saat halaman dibuka
Pindah kolom	`textInputAction: TextInputAction.next`	Tombol "berikutnya" di keyboard pindah ke kolom selanjutnya
Selesai dan simpan	`textInputAction: TextInputAction.done` + `onFieldSubmitted: (_) => _simpan()`	Tombol "selesai" langsung menyimpan
Fokus manual	`final _namaFocus = FocusNode();` lalu `_namaFocus.requestFocus()`	Kursor kembali ke Nama setelah form dikosongkan
Tutup keyboard	`FocusScope.of(context).unfocus()`	Keyboard turun saat Simpan ditekan

> FocusNode, seperti controller, wajib di-`dispose()`.

6.8 [Aplikatif] Merancang Aturan Validasi (PjBL)

Kriteria RPS pertemuan ini adalah **kelengkapan aturan validasi** dan **kemudahan penggunaan**. Sebelum menulis kode, susun tabel aturan seperti ini:

Kolom	Aturan	Pesan galat	Bantuan pengguna
Nama	Wajib; minimal 3 huruf	"Nama wajib diisi" / "Nama minimal 3 huruf"	Huruf besar otomatis; autofocus
NIM	Wajib; hanya angka; minimal 7 digit	"NIM wajib diisi" / "NIM hanya boleh berisi angka" / "NIM minimal 7 digit"	Keyboard angka; digitsOnly; maxLength 12
Email	Wajib; format email	"Email wajib diisi" / "Format email tidak valid (contoh: …)"	Keyboard email; hint contoh
Semester	Wajib; angka 1–14	"Semester wajib diisi" / "Semester harus antara 1 dan 14"	Keyboard angka; tombol "selesai" menyimpan

Tiga aturan pertama (wajib isi, NIM numerik, format email) diminta langsung oleh indikator RPS. Aturan panjang NIM dan rentang semester adalah contoh tambahan.

6.9 [Aplikatif] Menyambungkan ke Proyek

- Dashboard mendapat tombol **Tambah** (`FloatingActionButton.extended`) yang membuka `TambahMahasiswaPage` dengan `Navigator.push` (P4).

- Data yang valid ditampilkan sebagai **pratinjau** dan **SnackBar** (pesan singkat di bawah layar).
- Data **belum** masuk ke daftar Dashboard. Mengembalikan data ke Dashboard dan memperbarui daftar adalah materi Pertemuan 6.



Gambar 6.4 Rancangan form Tambah Mahasiswa dengan pesan galat

7. Contoh Kode

7.1 Kode lengkap (lib/main.dart)

Bagian baru: `import services.dart`, tombol Tambah di Dashboard, empat fungsi validator, dan class `TambahMahasiswaPage`. Bagian lain sama dengan Pertemuan 4.

```
// Pertemuan 5 - Form tambah data mahasiswa dengan validasi (Tugas 4)
// Proyek semester: Aplikasi Akademik Mahasiswa
import 'package:flutter/material.dart';
// services.dart: untuk FilteringTextInputFormatter (membatasi ketikan).
import 'package:flutter/services.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Aplikasi Akademik Mahasiswa',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
      ),
      // Halaman pertama sekarang Dashboard.
      home: const DashboardPage(),
    );
  }
}

// Data contoh: List berisi Map, satu Map untuk satu mahasiswa.
const List<Map<String, String>> daftarMahasiswa = [
  {
    'nama': 'Rina Amelia',
    'nim': '2023001',
    'prodi': 'Sistem Informasi',
    'semester': '5',
    'email': 'rina@contoh.ac.id',
  },
  {
    'nama': 'Budi Santoso',
    'nim': '2023002',
    'prodi': 'Sistem Informasi',
    'semester': '5',
    'email': 'budi@contoh.ac.id',
  },
  {
    'nama': 'Siti Rahma',
    'nim': '2023003',
    'prodi': 'Sistem Informasi',
    'semester': '3',
    'email': 'siti@contoh.ac.id',
  },
];

// ==================== HALAMAN 1: DASHBOARD ====================
class DashboardPage extends StatelessWidget {
  const DashboardPage({super.key});

  @override
```

```
Widget build(BuildContext context) {
  final warna = Theme.of(context).colorScheme;
  return Scaffold(
    appBar: AppBar(
      title: const Text('Dashboard Akademik'),
      backgroundColor: warna.primaryContainer,
    ),
    // Tombol menuju form tambah mahasiswa (baru di P5).
    floatingActionButton: FloatingActionButton.extended(
      onPressed: () {
        Navigator.push(
          context,
          MaterialPageRoute(builder: (context) => const TambahMahasiswaPage()),
        );
      },
      icon: const Icon(Icons.person_add),
      label: const Text('Tambah'),
    ),
    body: ListView(
      // Ruang bawah 88 agar kartu terakhir tidak tertutup tombol.
      padding: const EdgeInsets.fromLTRB(16, 16, 16, 88),
      children: [
        // Menu menuju halaman Profil (tanpa data).
        Card(
          child: ListTile(
            leading: const Icon(Icons.person),
            title: const Text('Profil Saya'),
            subtitle: const Text('Identitas dan keahlian'),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              // push: taruh halaman Profil di atas tumpukan.
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const ProfilPage()),
              );
            },
          ),
        ),
        const SizedBox(height: 16),
        const Text(
          'Daftar Mahasiswa',
          style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 8),
        // Satu kartu untuk setiap mahasiswa.
        for (final mhs in daftarMahasiswa)
          Card(
            child: ListTile(
              leading: CircleAvatar(child: Text(mhs['nama']![0])),
              title: Text(mhs['nama']!),
              subtitle: Text('NIM ${mhs['nim']}'),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {
                // Kirim data mahasiswa yang diketuk lewat constructor.
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => DetailMahasiswaPage(mahasiswa: mhs),
                  ),
                );
              },
            ),
          ),
      ],
    ),
  );
```

```
  }
}

// ============ ATURAN VALIDASI (fungsi biasa, mudah diuji) ============
// Aturan validator: kembalikan String (pesan galat) bila salah,
// atau null bila isian valid.

String? validasiNama(String? nilai) {
  final teks = nilai?.trim() ?? '';
  if (teks.isEmpty) return 'Nama wajib diisi';
  if (teks.length < 3) return 'Nama minimal 3 huruf';
  return null;
}

String? validasiNim(String? nilai) {
  final teks = nilai?.trim() ?? '';
  if (teks.isEmpty) return 'NIM wajib diisi';
  if (!RegExp(r'^[0-9]+$').hasMatch(teks)) {
    return 'NIM hanya boleh berisi angka';
  }
  // Panjang minimal hanya contoh; sesuaikan dengan format NIM kampus.
  if (teks.length < 7) return 'NIM minimal 7 digit';
  return null;
}

String? validasiEmail(String? nilai) {
  final teks = nilai?.trim() ?? '';
  if (teks.isEmpty) return 'Email wajib diisi';
  final polaEmail = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$');
  if (!polaEmail.hasMatch(teks)) {
    return 'Format email tidak valid (contoh: nama@kampus.ac.id)';
  }
  return null;
}

String? validasiSemester(String? nilai) {
  final teks = nilai?.trim() ?? '';
  if (teks.isEmpty) return 'Semester wajib diisi';
  final angka = int.tryParse(teks);
  if (angka == null) return 'Semester harus berupa angka';
  // Batas 14 hanya contoh; sesuaikan dengan peraturan akademik.
  if (angka < 1 || angka > 14) return 'Semester harus antara 1 dan 14';
  return null;
}

// ================ HALAMAN 4: FORM TAMBAH MAHASISWA ================
// StatefulWidget karena isi form, mode validasi, dan hasil berubah.
class TambahMahasiswaPage extends StatefulWidget {
  const TambahMahasiswaPage({super.key});

  @override
  State<TambahMahasiswaPage> createState() => _TambahMahasiswaPageState();
}

class _TambahMahasiswaPageState extends State<TambahMahasiswaPage> {
  // Kunci untuk memanggil validate() dan reset() pada Form.
  final _formKey = GlobalKey<FormState>();

  // Controller: membaca dan mengatur isi setiap kolom.
  final _namaController = TextEditingController();
  final _nimController = TextEditingController();
  final _emailController = TextEditingController();
  final _semesterController = TextEditingController();
```

```
// FocusNode: mengatur kolom mana yang sedang aktif.
final _namaFocus = FocusNode();

// Pesan galat otomatis baru aktif setelah Simpan pertama ditekan.
AutovalidateMode _modeValidasi = AutovalidateMode.disabled;

// Data yang lolos validasi, ditampilkan sebagai pratinjau.
Map<String, String>? _hasil;

@override
void dispose() {
  // Wajib: lepaskan controller dan FocusNode saat halaman ditutup.
  _namaController.dispose();
  _nimController.dispose();
  _emailController.dispose();
  _semesterController.dispose();
  _namaFocus.dispose();
  super.dispose();
}

void _simpan() {
  FocusScope.of(context).unfocus(); // tutup keyboard
  if (_formKey.currentState!.validate()) {
    setState(() {
      _hasil = {
        'nama': _namaController.text.trim(),
        'nim': _nimController.text.trim(),
        'email': _emailController.text.trim(),
        'semester': _semesterController.text.trim(),
      };
    });
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Data ${_hasil!['nama']} valid.')),
    );
  } else {
    setState(() {
      _modeValidasi = AutovalidateMode.onUserInteraction;
    });
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Periksa kembali isian yang ditandai merah.'),
      ),
    );
  }
}

void _kosongkan() {
  _formKey.currentState!.reset();
  _namaController.clear();
  _nimController.clear();
  _emailController.clear();
  _semesterController.clear();
  setState(() {
    _hasil = null;
    _modeValidasi = AutovalidateMode.disabled;
  });
  _namaFocus.requestFocus(); // kursor kembali ke kolom pertama
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Tambah Mahasiswa')),
```

```
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          autovalidateMode: _modeValidasi,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              // 1. Nama
              TextFormField(
                controller: _namaController,
                focusNode: _namaFocus,
                autofocus: true,
                textCapitalization: TextCapitalization.words,
                textInputAction: TextInputAction.next,
                decoration: const InputDecoration(
                  labelText: 'Nama lengkap',
                  hintText: 'mis. Rina Amelia',
                  prefixIcon: Icon(Icons.person),
                  border: OutlineInputBorder(),
                ),
                validator: validasiNama,
              ),
              const SizedBox(height: 16),
              // 2. NIM: keyboard angka dan hanya menerima digit.
              TextFormField(
                controller: _nimController,
                keyboardType: TextInputType.number,
                textInputAction: TextInputAction.next,
                inputFormatters: [FilteringTextInputFormatter.digitsOnly],
                maxLength: 12,
                decoration: const InputDecoration(
                  labelText: 'NIM',
                  hintText: 'hanya angka',
                  prefixIcon: Icon(Icons.badge),
                  border: OutlineInputBorder(),
                ),
                validator: validasiNim,
              ),
              const SizedBox(height: 8),
              // 3. Email
              TextFormField(
                controller: _emailController,
                keyboardType: TextInputType.emailAddress,
                textInputAction: TextInputAction.next,
                decoration: const InputDecoration(
                  labelText: 'Email',
                  hintText: 'nama@kampus.ac.id',
                  prefixIcon: Icon(Icons.email),
                  border: OutlineInputBorder(),
                ),
                validator: validasiEmail,
              ),
              const SizedBox(height: 16),
              // 4. Semester: tombol "selesai" di keyboard langsung menyimpan.
              TextFormField(
                controller: _semesterController,
                keyboardType: TextInputType.number,
                textInputAction: TextInputAction.done,
                inputFormatters: [FilteringTextInputFormatter.digitsOnly],
                maxLength: 2,
                decoration: const InputDecoration(
                  labelText: 'Semester',
                  hintText: '1 sampai 14',
                  prefixIcon: Icon(Icons.school),
                  border: OutlineInputBorder(),
```

```
              ),
              validator: validasiSemester,
              onFieldSubmitted: (_) => _simpan(),
            ),
            const SizedBox(height: 16),
            FilledButton.icon(
              onPressed: _simpan,
              icon: const Icon(Icons.save),
              label: const Text('Simpan'),
            ),
            const SizedBox(height: 8),
            OutlinedButton.icon(
              onPressed: _kosongkan,
              icon: const Icon(Icons.refresh),
              label: const Text('Kosongkan'),
            ),
            // Pratinjau hanya tampil setelah data valid.
            if (_hasil != null) ...[
              const SizedBox(height: 24),
              Container(
                padding: const EdgeInsets.all(16),
                decoration: BoxDecoration(
                  color: Theme.of(context).colorScheme.primaryContainer,
                  borderRadius: BorderRadius.circular(16),
                ),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    const Text(
                      'Pratinjau data valid',
                      style: TextStyle(fontWeight: FontWeight.bold),
                    ),
                    const SizedBox(height: 8),
                    BarisInfo(label: 'Nama', nilai: _hasil!['nama']!),
                    BarisInfo(label: 'NIM', nilai: _hasil!['nim']!),
                    BarisInfo(label: 'Email', nilai: _hasil!['email']!),
                    BarisInfo(label: 'Semester', nilai: _hasil!['semester']!),
                  ],
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

// ================= HALAMAN 2: DETAIL MAHASISWA =================
class DetailMahasiswaPage extends StatelessWidget {
  // required: halaman ini tidak bisa dibuka tanpa data mahasiswa.
  const DetailMahasiswaPage({super.key, required this.mahasiswa});

  final Map<String, String> mahasiswa;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // AppBar otomatis menampilkan tombol kembali (←).
      appBar: AppBar(title: const Text('Detail Mahasiswa')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
```

```
          children: [
            BarisInfo(label: 'Nama', nilai: mahasiswa['nama'] ?? '-'),
            BarisInfo(label: 'NIM', nilai: mahasiswa['nim'] ?? '-'),
            BarisInfo(label: 'Program Studi', nilai: mahasiswa['prodi'] ?? '-'),
            BarisInfo(label: 'Semester', nilai: mahasiswa['semester'] ?? '-'),
            BarisInfo(label: 'Email', nilai: mahasiswa['email'] ?? '-'),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: () {
                // pop: buang halaman ini, kembali ke Dashboard.
                Navigator.pop(context);
              },
              icon: const Icon(Icons.arrow_back),
              label: const Text('Kembali ke Dashboard'),
            ),
          ],
        ),
      ),
    );
  }
}

// ================== HALAMAN 3: PROFIL (dari P3) ==================
class ProfilPage extends StatelessWidget {
  const ProfilPage({super.key});

  static const Map<String, String> profil = {
    'Nama': 'Rina Amelia',
    'NIM': '2023001',
    'Program Studi': 'Sistem Informasi',
    'Semester': '5',
    'Email': 'rina@contoh.ac.id',
  };

  static const List<String> keahlian = ['Flutter', 'Basis Data', 'Desain UI'];

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Profil Mahasiswa'),
        backgroundColor: warna.primaryContainer,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Container(
              padding: const EdgeInsets.all(16),
              decoration: BoxDecoration(
                color: warna.primaryContainer,
                borderRadius: BorderRadius.circular(16),
              ),
              child: Row(
                children: [
                  Container(
                    width: 88,
                    height: 88,
                    clipBehavior: Clip.antiAlias,
                    decoration: const BoxDecoration(
                      shape: BoxShape.circle,
                      color: Colors.white,
                    ),
```

```
                    child: Image.asset(
                      'assets/images/foto.jpg',
                      fit: BoxFit.cover,
                      errorBuilder: (context, error, stackTrace) =>
                          const Icon(Icons.person, size: 56),
                    ),
                  ),
                  const SizedBox(width: 16),
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          profil['Nama']!,
                          style: const TextStyle(
                            fontSize: 20,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        const SizedBox(height: 4),
                        Text('${profil['NIM']} · ${profil['Program Studi']}'),
                      ],
                    ),
                  ),
                ],
              ),
            ),
            const SizedBox(height: 24),
            const Text(
              'Identitas',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 8),
            for (final data in profil.entries)
              BarisInfo(label: data.key, nilai: data.value),
            const SizedBox(height: 24),
            const Text(
              'Keahlian',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 8),
            for (final item in keahlian)
              Container(
                padding: const EdgeInsets.symmetric(vertical: 4),
                child: Row(
                  children: [
                    Icon(Icons.check_circle, color: warna.primary),
                    const SizedBox(width: 8),
                    Text(item),
                  ],
                ),
              ),
          ],
        ),
      ),
    );
  }
}

// Widget baris "label : nilai" (dari P3), dipakai di Profil dan Detail.
class BarisInfo extends StatelessWidget {
  const BarisInfo({super.key, required this.label, required this.nilai});

  final String label;
  final String nilai;
```

```
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(vertical: 6),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(
            width: 120,
            child: Text(
              label,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
          ),
          const Text(': '),
          Expanded(child: Text(nilai)),
        ],
      ),
    );
  }
}
```

7.2 Penjelasan baris penting

Baris kode	Arti
`import 'package:flutter/services.dart';`	Pustaka bawaan Flutter untuk `FilteringTextInputFormatter`.
`String? validasiNim(String? nilai)`	Validator sebagai fungsi biasa: menerima isian, mengembalikan pesan atau null.
`class TambahMahasiswaPage extends StatefulWidget`	Form butuh state: isian, mode validasi, dan hasil.
`final _formKey = GlobalKey<FormState>();`	Kunci untuk memanggil `validate()` dan `reset()`.
`final _namaController = TextEditingController();`	Satu controller per kolom, dibuat sekali di State.
`void dispose() { ... }`	Melepas controller dan FocusNode saat halaman ditutup.
`if (_formKey.currentState!.validate())`	Memeriksa semua kolom sekaligus.
`_modeValidasi = AutovalidateMode.onUserInteraction`	Setelah gagal sekali, pesan diperbarui saat pengguna mengetik.
`ScaffoldMessenger.of(context).showSnackBar(...)`	Menampilkan pesan singkat di bawah layar.
`inputFormatters: [FilteringTextInputFormatter.digitsOnly]`	Hanya angka yang bisa diketik.
`onFieldSubmitted: (_) => _simpan()`	Tombol "selesai" di keyboard menjalankan Simpan.
`if (_hasil != null) ...[ ... ]`	Pratinjau hanya tampil bila ada data valid. `...` menyisipkan beberapa widget sekaligus.

7.3 Uji otomatis (test/widget_test.dart)

Berisi uji alur P4 (diringkas), uji form, dan uji fungsi validator. Fungsi validator bisa diuji tanpa tampilan karena ditulis terpisah.

```
// test/widget_test.dart - uji alur (P4) + uji form dan aturan validasi (P5)
import 'package:flutter_test/flutter_test.dart';

import 'package:aplikasi_akademik/main.dart';

void main() {
  testWidgets('Dashboard membuka Detail dengan data yang benar',
      (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.tap(find.text('Budi Santoso'));
    await tester.pumpAndSettle();
    expect(find.text('2023002'), findsOneWidget);
  });

  testWidgets('Form menampilkan pesan galat', (WidgetTester tester) async {
    await tester.pumpWidget(const MyApp());
    await tester.tap(find.text('Tambah'));
    await tester.pumpAndSettle();

    // Isi nama dan email yang salah, NIM dibiarkan kosong.
    await tester.enterText(find.byType(TextFormField).at(0), 'Rina Amelia');
    await tester.enterText(find.byType(TextFormField).at(2), 'rina@');
    await tester.tap(find.text('Simpan'));
    await tester.pump();

    expect(find.text('NIM wajib diisi'), findsOneWidget);
    expect(find.textContaining('Format email tidak valid'), findsOneWidget);
  });

  test('Aturan validasi NIM dan email', () {
    expect(validasiNim(''), 'NIM wajib diisi');
    expect(validasiNim('2023a01'), 'NIM hanya boleh berisi angka');
    expect(validasiNim('123'), 'NIM minimal 7 digit');
    expect(validasiNim('2023001'), isNull);
    expect(validasiEmail('rina@contoh.ac.id'), isNull);
    expect(validasiEmail('rina.contoh'), isNotNull);
  });
}
```

7.4 Kode review StatefulWidget

Kode sapaan di Bab 6.1 dijalankan di DartPad, terpisah dari proyek.

7.5 Perbandingan dengan Pertemuan 4

Yang berubah	Yang tetap
Import `services.dart`; tombol Tambah di Dashboard; padding bawah daftar; validator; `TambahMahasiswaPage`; uji form	`MyApp`, data `daftarMahasiswa`, `DetailMahasiswaPage`, `ProfilPage`, `BarisInfo`

7.6 Bagian yang perlu diuji dosen

> Penyusun **tidak dapat menjalankan Flutter**. Mohon uji di lab: (1) jalankan `flutter analyze` dan `flutter test`; (2) coba form di emulator/HP: urutan fokus, keyboard angka, tombol "selesai", dan

> Kosongkan. Bagian yang paling perlu dicek: perlu tidaknya `import 'package:flutter/services.dart'` (bila analyzer menandai *unnecessary import*, baris itu boleh dihapus); `Icons.badge`; letak tombol Simpan di layar uji (800×600) untuk `tester.tap`; dan perilaku `reset()` pada kolom yang memakai controller.

8. Praktikum Terpandu

Tujuan: proyek `aplikasi_akademik` memiliki form Tambah Mahasiswa yang lengkap validasinya dan nyaman dipakai. **Waktu:** Bagian A ±15 menit, Bagian B–E ±45 menit (PjBL).

Bagian A – Review StatefulWidget (DartPad)

No	Perintah / aksi	Hasil yang harus tampil	Tanda berhasil
1	Buka dartpad.dev, tempel kode sapaan Bab 6.1, klik Run.	Kolom isian dan teks "Halo!".	—
2	Ketik nama Anda.	Teks berubah setiap huruf: "Halo, Rina!".	Berubah langsung.
3	Hapus `setState` sehingga baris menjadi `onChanged: (teks) => _nama = teks;`. Run, lalu ketik.	Teks **tidak** berubah.	Anda bisa menjelaskan alasannya.

Bagian B – Rancang aturan

No	Perintah / aksi	Hasil yang harus tampil	Tanda berhasil
4	Lengkapi tabel aturan validasi (Bab 6.8) di kertas untuk form Anda. Minimal: wajib isi, NIM numerik, format email.	Tabel kolom–aturan–pesan–bantuan.	Diparaf dosen/asisten.

Bagian C – Halaman form dan navigasi

No	Perintah / aksi	Hasil yang harus tampil	Tanda berhasil
5	Tambahkan `import 'package:flutter/services.dart';` dan empat fungsi validator di `main.dart`.	Tidak ada garis merah.	—
6	Buat class `TambahMahasiswaPage` (StatefulWidget) berisi Scaffold dan AppBar "Tambah Mahasiswa".	—	—
7	Tambahkan `floatingActionButton` di Dashboard yang membuka halaman itu. Simpan.	Tombol "Tambah" di kanan bawah Dashboard. [Tangkapan layar]	Mengetuknya membuka halaman form.

Bagian D – Isi form

No	Perintah / aksi	Hasil yang harus tampil	Tanda berhasil
8	Di State: buat `_formKey`, empat controller, `_namaFocus`, dan method `dispose()`.	—	Setiap controller punya pasangan `dispose()`.
9	Di `build`: `SingleChildScrollView` → `Form(key: _formKey)` → `Column` berisi empat `TextFormField` dengan `validator`.	Empat kolom berlabel dan berikon.	—

No	Perintah / aksi	Hasil yang harus tampil	Tanda berhasil
10	Tambahkan tombol Simpan yang memanggil `_simpan()`. Tekan Simpan dengan semua kolom kosong.	Empat pesan merah dan SnackBar "Periksa kembali …". [Tangkapan layar]	Semua pesan "… wajib diisi" tampil.
11	Isi NIM `2023a01` dengan menempel (*paste*) teks, dan email `rina@`. Simpan.	Pesan "NIM hanya boleh berisi angka" dan "Format email tidak valid …".	Pesan sesuai aturan.
12	Perbaiki isian satu per satu.	Pesan merah hilang begitu kolom benar (autovalidateMode).	—
13	Isi semua dengan benar, tekan Simpan.	SnackBar "Data … valid." dan kotak pratinjau. [Tangkapan layar]	Data pratinjau sama dengan isian.

Bagian E – Kemudahan penggunaan, uji, kirim

No	Perintah / aksi	Hasil yang harus tampil	Tanda berhasil
14	Buka form baru. Ketik nama tanpa mengetuk kolom.	Kursor sudah di kolom Nama (autofocus).	—
15	Tekan tombol "berikutnya" di keyboard sampai Semester, lalu tombol "selesai".	Fokus berpindah berurutan; "selesai" menjalankan Simpan.	Tanpa mengetuk kolom satu per satu.
16	Cek jenis keyboard pada NIM dan Email.	Keyboard angka untuk NIM; keyboard dengan @ untuk Email.	—
17	Tekan Kosongkan.	Semua kolom kosong, pesan hilang, kursor kembali ke Nama.	—
18	Ganti `test/widget_test.dart` dengan Bab 7.3. Jalankan `flutter analyze` dan `flutter test`.	`No issues found!` dan `All tests passed!`	—
19	Tukar perangkat dengan teman: isi form teman dengan data salah, catat 1 saran kemudahan.	Lembar peer review.	—
20	`git add .` → `git commit -m "Pertemuan 5: form tambah mahasiswa"` → `git push`.	Commit baru di GitHub.	—

Alternatif

Kondisi	Solusi
Tanpa HP Android / RAM kecil	`flutter run -d chrome`. Keyboard fisik dipakai; tombol Tab berpindah kolom, Enter pada kolom terakhir menyimpan.
Proyek bermasalah	Tempel kode 7.1 di DartPad. Semua fitur form bisa dicoba.
Emulator tanpa keyboard di layar	Aktifkan keyboard virtual di pengaturan emulator, atau pakai keyboard laptop.

9. Kesalahan Umum Pemula

Teks pesan dapat sedikit berbeda antarversi Dart dan Flutter.

Gejala	Pesan error asli	Penyebab	Solusi
Aplikasi berhenti saat Simpan	`Null check operator used on a null value`	`_formKey` belum dipasang di `Form(key: _formKey)`, sehingga `currentState` null.	Pasang `key: _formKey` pada Form.
setState tidak dikenal	`The method 'setState' isn't defined for the type '...'.`	Memanggil `setState` di StatelessWidget.	Ubah menjadi StatefulWidget.
controller dan initialValue bersamaan	Mis. `'initialValue == null || controller == null': is not true.`	TextFormField diberi keduanya.	Pakai salah satu; isi nilai awal lewat `controller.text`.
Formatter tidak dikenal	`Undefined name 'FilteringTextInputFormatter'.`	Belum mengimpor `services.dart`.	Tambahkan `import 'package:flutter/services.dart';`.
Tidak bisa simpan, tanpa pesan	— (tidak ada galat)	Validator mengembalikan `''` (string kosong). Dianggap tidak valid, tetapi pesannya kosong.	Kembalikan `null` bila valid; pesan jelas bila tidak.
Email benar selalu ditolak	— (tidak ada galat)	Pola RegExp ditulis tanpa `r`, sehingga `\w` terbaca sebagai huruf w.	Gunakan *raw string* `r'...'`.
Isian hilang atau kursor melompat	—	Controller dibuat di dalam `build()`.	Buat controller sebagai field State.
Pita kuning-hitam saat keyboard muncul	`A RenderFlex overflowed by … pixels on the bottom.`	Kolom tidak bisa digulir saat keyboard menutup layar.	Bungkus dengan `SingleChildScrollView`.
SnackBar gagal	Mis. `No ScaffoldMessenger widget found.`	Halaman diuji/ditampilkan tanpa `MaterialApp` di atasnya.	Pastikan halaman berada di bawah `MaterialApp`.
Spasi lolos sebagai nama	—	Validator memeriksa `isEmpty` tanpa `trim()`.	`nilai?.trim() ?? ''`.
Pesan merah sudah muncul sebelum mengetik	—	`autovalidateMode: AutovalidateMode.always`.	Mulai dengan `disabled`, ubah ke `onUserInteraction` setelah Simpan gagal.

Gejala	Pesan error asli	Penyebab	Solusi
Perubahan validator tidak terasa	—	State lama masih tersimpan.	Hot Restart (R) atau buka ulang halaman form.

10. Latihan Mandiri Bertingkat

Mudah: validator nomor HP

Tulis fungsi `String? validasiNoHp(String? nilai)` dengan aturan: wajib isi; hanya angka; diawali `08`; panjang 10–13 digit. Uji di DartPad dengan `print(validasiNoHp('0812'))` dan beberapa contoh lain.

Petunjuk: `teks.startsWith('08')` dan `teks.length`.

Sedang: tambah kolom No. HP dan Program Studi

1. Tambahkan dua `TextFormField` baru ke form: No. HP (pakai `validasiNoHp`) dan Program Studi (wajib isi).
2. Atur `keyboardType`, `textInputAction`, ikon, dan hint yang sesuai.
3. Pastikan urutan fokus tetap benar dan kolom terakhir yang memakai `done`.
4. Tambahkan keduanya ke pratinjau.

Tantangan: form pengisian KRS (tema sistem informasi)

Bagian akademik menerapkan aturan: mahasiswa dengan IPK ≥ 3,00 boleh mengambil maksimal 24 SKS; di bawah itu maksimal 20 SKS. Buat halaman form KRS berisi NIM, IPK semester lalu, dan jumlah SKS yang diambil.

1. IPK: wajib, desimal 0,00–4,00 (petunjuk: `double.tryParse`, terima titik sebagai pemisah desimal).
2. SKS: wajib, bilangan bulat, dan **tidak melebihi batas sesuai IPK**. Ini validasi lintas kolom: validator SKS membaca nilai controller IPK.
3. Pesan galat harus menyebut batas yang berlaku, mis. "IPK 2,80: maksimal 20 SKS".
4. Tulis tabel aturan sebelum menulis kode.

Ambang IPK dan SKS di soal ini fiktif; sesuaikan dengan peraturan kampus bila ingin realistis.

11. Kuis (5 Soal)

1. Halaman form menyimpan isian dan hasil validasi yang berubah-ubah. Jenis widget yang tepat untuk halaman ini adalah...

A. StatelessWidget, karena tampilannya tetap
B. Container, karena bisa memuat banyak kolom
C. StatefulWidget, karena datanya berubah dan perlu setState()
D. Scaffold, karena memiliki AppBar

2. Sebuah validator mengembalikan `null`. Artinya...

A. Isian kolom valid
B. Isian kolom kosong
C. Terjadi galat pada aplikasi
D. Form harus di-reset

3. Apa hasil `validasiNim('12a45')` pada kode Bab 7?

A. null
B. 'NIM wajib diisi'
C. 'NIM minimal 7 digit'
D. 'NIM hanya boleh berisi angka'

4. Fungsi `GlobalKey<FormState>` pada form adalah...

A. Menyimpan isi setiap kolom
B. Memberi akses ke state Form agar bisa memanggil validate() dan reset()
C. Mengatur urutan fokus kolom
D. Mengenkripsi data form

5. Pengguna mengisi semua kolom dengan benar dan menekan Simpan, tetapi data tidak pernah dianggap valid dan tidak ada pesan galat yang tampil. Penyebab paling mungkin adalah...

A. autofocus belum diaktifkan
B. keyboardType salah
C. Salah satu validator mengembalikan string kosong '', bukan null
D. Controller belum di-dispose